EX13 - Trayectorias con listas y tuplas - Guardar el historial

Guia paso a paso para principiantes

Esta guia sigue el archivo real `ex13_trajectory_lists_tuples.py`. Se explica primero el problema y despues el codigo desde el punto de vista de alguien que esta empezando Python.

1. Que queremos resolver

Modificar la simulacion para conservar todos los estados de la trayectoria, no solo el estado actual y el resultado final.

Variables / elementos del problema

Nombre	Significado
state	(h,v,t) en un instante
trajectory	lista de estados

Ecuacion o logica en papel

```
state_k = (h_k, v_k, t_k)
trajectory = [state_0, state_1, ...]
```

Comprobacion manual

Tras el primer paso de Euler: state_0=(0,50,0) y state_1=(5.0,49.019,0.1). trajectory debe contener ambos.

2. Lo que ya sabemos y lo nuevo

Ya visto:

- funciones
- while
- Euler
- return multiple

Nuevo en este ejercicio:

- list
- []
- append()
- tuple
- indices [i] y [-1]
- generator expression dentro de max()
- snapshots

3. Conceptos Python nuevos

1. Lista

Sintaxis: `trajectory = []`

Una lista es ordenada y mutable. Puede empezar vacia y crecer durante la simulacion.

2. append

Sintaxis: `trajectory.append((h, v, t))`

append añade un unico elemento al final. Aqui el elemento es una tuple.

3. Tuple

Sintaxis: `(h, v, t)`

Usamos una tuple como snapshot compacto del estado.

4. Indice negativo

Sintaxis: `trajectory[-1]`

-1 selecciona el ultimo elemento de una secuencia.

5. Expresion generadora

Sintaxis: `max(state[0] for state in trajectory)`

Genera state[0] de cada estado sin crear manualmente otra lista.

4. Codigo paso a paso

Paso 1. Crear listas vacias

```
h = []
v = []
t = []
trajectory = []
```

Las colecciones empiezan vacias y se rellenan con append().

Paso 2. Guardar estado inicial

```
h.append(h0)
v.append(v0)
t.append(t0)
trajectory.append((h0, v0, t0))
```

Guardamos el estado inicial para que tambien forme parte del historial.

Paso 3. Acceder al estado actual

```
while h[i] >= 0:
    h_new, v_new, t_new = euler_step(h[i], v[i], t[i], g, dt)
```

Como h,v,t son listas, i indica que muestra es el estado actual.

Paso 4. Anadir un nuevo snapshot

```
trajectory.append((h_new, v_new, t_new))
```

Cada iteracion crea una tuple nueva, por lo que los estados anteriores se conservan.

Paso 5. Extraer resultados

```
max_h_trajectory = max(state[0] for state in trajectory)
v_f_trajectory = trajectory[-1][1]
t_f_trajectory = trajectory[-1][2] - t0
```

state[0] es altura, [1] velocidad y [2] tiempo. trajectory[-1] es el ultimo estado.

5. Flujo de ejecucion

```
initialize lists
 -> append state_0
 -> Euler loop
 -> append each state
 -> analyze trajectory
```

6. Resultado / comprobacion esperada

Los valores extraidos de trajectory deben coincidir con los resultados directos: max_h~129.92 m, t_f~10.30 s y v_f~-51.04 m/s.

7. Errores tipicos al empezar

- Confundir append(x) con varios argumentos.
- Guardar siempre el mismo objeto mutable como estado.
- Confundir trajectory[-1][1] con trajectory[1][-1].
- Olvidar que los indices empiezan en 0.

8. Si solo recuerdas cinco cosas

- [] crea una lista vacia.
- append anade un elemento.
- Una tuple (h,v,t) representa un estado.
- -1 significa ultimo.
- Guardar historial permite postprocesar la simulacion.

9. Mini practica

Cambia un unico input. Antes de ejecutar, escribe en un comentario que esperas que cambie y por que. Despues ejecuta y compara la prediccion con la salida.

Criterio de cierre

Puedes continuar cuando puedas explicar el problema, la ecuacion/logica, los conceptos Python nuevos y el flujo de ejecucion sin limitarte a leer el codigo.